

Learning from Agentic Traces

Tommaso Furlanello @cyndesama

tags: @vision @abstractions

We characterize a Partially Observable Markov Decision Process (POMDP) as an autoregressive stochastic process over actions and observations. We use this formulation as a generator for agentic traces and derive what information is stored in a model trained to autoregressively predict sequences sampled from such a generator. Concretely, we treat turn-labeled sequences as the observable process and the POMDP as the latent mechanism that produces them. We build intuition about what Large Language Models (LLMs) would implicitly learn during pretraining if trained on an extensive corpus of agentic traces, under the simple assumption that the tokenization space aligns with the action and observation spaces.

The POMDP Frame or When is a System Agentic?

We start by asking in what sense a dynamical system can be said to be agentic, and what latent structure can be recovered from the system’s measurable behavior when this assumption holds. Agency developed historically as a model of what separates humans from the rest of physical matter, and more recently as a model of what makes a system intelligent. Being humans, we inherently perceive a separation between what we are as a mind, the agent, and the thing-in-itself we live in, the environment. This dualism seems to clash with our understanding of reality as a local physical process, yet it is reconciled in modern science as a conditional independence, or causal, structure that appears when we measure the world at a coarse enough scale. If we zoom out sufficiently, we can see how the states inside the agent are causally independent of the states in the environment given the information that leaks in at the boundary between them. We can think of this as the agent’s perceptual channel. Conversely, we can see how the state of the world is conditionally independent of what goes on inside the agent given the information that leaks out at the boundary, which we can call the causal or action channel.

Make the physical example concrete. Imagine a room with a robotic arm at a keyboard connected to an Atari console, a Raspberry Pi running the controller, and a camera watching the Atari’s monitor. We can leave the robot in the room until it learns to play the game at the best of its capacity, then we start measuring. If all we can monitor in that room are keypress events and camera frames, those two streams are our entire window into the dynamics at this resolution. Continuous time effectively unfolds in turns: a keypress is an action, the game produces pixels as a response, and the cycle repeats.

Given these measurements, we could be tempted to build a complete bottom-up model of the room, starting from silicon steering electrons through transistors, up to the motor drivers and linkages that move the arm, and down again to the specific software implementation of the game and the hardware it runs on. That would be a daunting task, and it would change if the same behavioral input were generated by a different robot or if the game were implemented on different hardware.

Thankfully, none of these details change the coarse-grained causal boundary between agent and environment. Given the keys that were pressed, the game’s next internal update does not depend on the Pi’s microstate; given the pixels that hit the camera, the Pi’s next update does not depend on the game’s microphysics. The decomposition between agent and environment gives us the structure to describe a stochastic generator of the measurement process that is invariant to the microscopic details of the system.

In this sense, it is useful to speak about an agentic system when its measurable behavior can be decomposed into two components: actions and observations, each defining a clear causal boundary, or Markov blanket, between the environment and the agent. In this section, we formally derive the POMDP frame as a decomposition of an autoregressive stochastic process into two stationary components. This setup allows us to connect the dots with our understanding of LLMs as approximators of autoregressive stochastic processes, yielding insights into what an LLM trained on agentic traces would learn.

Agentic Traces as an Autoregressive Stochastic Process

To make our discussion formal, we start from reviewing the general language of POMDPs to define the macro-level agent-environment interaction. In practice modelling a system as a POMDP corresponds to assuming a specific decomposition of the observable process, or agentic-trace, Z into a sequence of turns $Z = \{z_1, z_2, \dots, z_T\}$, where each turn $z_t \in \mathcal{Z}$ is a tuple (a_t, o_t) of an action a_t and an observation o_t . Intuitively the action a_t in $\mathcal{A}$ defines the component of the observable turn that is causally controlled by the agent, while the observation o_t in $\mathcal{O}$ defines the component that is causally controlled by the environment conditioned on the agent behavior. The sets $\mathcal{A}$ and $\mathcal{O}$ decompose the degrees of freedom of $z \in \mathcal{Z}$ into the product $\mathcal{A} \times \mathcal{O}$.

Then we assume that the probability of the next joint turn $z_{t+1} = (a_{t+1}, o_{t+1})$ is only conditioned on the current environment and agent hidden states, respectively s_t and agent’s $\hat{s}_t$, and the agent’s parameters θ , defining a joint stochastic process with emissions $P(z_{t+1} | s_t, \hat{s}_t; \theta)$ whose temporal dynamics are modeled by the recurrent transition $P(s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t, z_{t+1}; \theta)$. Or explicitly with respect to actions and observations $P(a_{t+1}, o_{t+1} | s_t, \hat{s}_t; \theta)$. In a POMDP it is possible to further decompose this joint process into the environment’s emission kernel $P(o_{t+1} | s_t, a_{t+1})$ and its emission-conditioned transition kernel, $P(s_{t+1} | s_t, a_{t+1}, o_{t+1})$, together with the agent’s policy $\pi(a_{t+1} | \hat{s}_t; \theta_\pi)$ and the agent’s state transition kernel $\mathcal{M}(\hat{s}_{t+1} | \hat{s}_t, a_{t+1}, o_{t+1}; \theta_\mathcal{M})$, with parameters $\theta = (\theta_\pi, \theta_\mathcal{M})$. This choice yields a unifilar latent update at the environment level: once s_t , a_{t+1} , and the realized o_{t+1} are known, the distribution over s_{t+1} is conditionally concentrated along a single causal branch consistent with that symbol.

The emission of the next turn decomposes into agent action selection and environment observation emission:

$$P(z_{t+1} | s_t, \hat{s}_t; \theta) = P(a_{t+1}, o_{t+1} | s_t, \hat{s}_t; \theta) = \pi(a_{t+1} | \hat{s}_t; \theta_\pi) \cdot P(o_{t+1} | s_t, a_{t+1}).$$

This is the macro interface where the agent proposes a_{t+1} and the environment commits to a symbol o_{t+1} that will also drive the unifilar update of its hidden state.

Given the emitted turn, both hidden states update according to their respective transition kernels:

$$P(s_{t+1}, \hat{s}_{t+1} | s_t, \hat{s}_t, z_{t+1}; \theta) = P(s_{t+1} | s_t, a_{t+1}, o_{t+1}) \cdot \mathcal{M}(\hat{s}_{t+1} | \hat{s}_t, a_{t+1}, o_{t+1}; \theta_\mathcal{M}).$$

Here the unifilar dependence of s_{t+1} on (s_t, a_{t+1}, o_{t+1}) guarantees that the latent path is synchronizable from the turn process in the sense used in computational mechanics.

Combining emission and transition dynamics, the complete joint update for each turn $z_{t+1} = (a_{t+1}, o_{t+1})$ becomes:

$$P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta) = P(z_{t+1} \mid s_t, \hat{s}_t; \theta) \cdot P(s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t, z_{t+1}; \theta).$$

Which expands to the full factorization $P(a_{t+1}, o_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta)$:

$$\pi(a_{t+1} \mid \hat{s}_t; \theta_\pi) \cdot P(o_{t+1} \mid s_t, a_{t+1}) \cdot P(s_{t+1} \mid s_t, a_{t+1}, o_{t+1}) \cdot \mathcal{M}(\hat{s}_{t+1} \mid \hat{s}_t, a_{t+1}, o_{t+1}; \theta_{\mathcal{M}}).$$

For a complete trajectory $Z = \{z_1, z_2, \dots, z_T\}$ with $z_t = (a_t, o_t)$, the joint probability over all turns and hidden states given the agent parameters θ is:

$$P(Z, s_{1:T}, \hat{s}_{1:T} \mid \theta) = P(s_1, \hat{s}_1) \cdot P(z_1 \mid s_1, \hat{s}_1; \theta) \cdot \prod_{t=1}^{T-1} P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta).$$

For a given agent, marginalizing over the hidden states gives the observable sequence probability:

$$P(Z \mid \theta) = \sum_{s_{1:T}, \hat{s}_{1:T}} P(s_1, \hat{s}_1) \cdot P(z_1 \mid s_1, \hat{s}_1; \theta) \cdot \prod_{t=1}^{T-1} P(z_{t+1}, s_{t+1}, \hat{s}_{t+1} \mid s_t, \hat{s}_t; \theta).$$

With the latest formulation we can effectively define the likelihood of our data conditional on a generative model that can be effectively compartmentalized into a parametrized agent and a stationary environment, that is an agentic system.

Learning from a Dataset of Agentic Traces

In the previous section we showed how a partially observable dynamical system can be decomposed into two stationary parts: a fixed environment and a parameterized agent. Now let's develop an intuition for what can be learned from data generated by that process. Assume the agent has converged to a stationary policy and we can sample a large dataset of agentic traces $Z = \{z_1, \dots, z_T\}$ with turns $z_t = (a_t, o_t)$. The question is simple: what is the informational relationship between the generator's hidden states $(s_t, \hat{s}_t)$ that produced a given z_t and the internal state $h_{\theta_G}(z_{<t})$ of a strong autoregressive predictor $G(\hat{z}_t \mid z_{<t}; \theta_G)$?

There are two standard ways to model hidden stochastic processes.

Belief over latent state. We put a probabilistic model on the world and filter observations into a belief over the joint latent state

$$b_t(s, \hat{s}) = P(s_t = s, \hat{s}_t = \hat{s} \mid a_{1:t}, o_{1:t})$$

After a new turn (a_{t+1}, o_{t+1}) the belief updates with Bayes:

$$b_{t+1}(s', \hat{s}') \propto \sum_{s, \hat{s}} b_t(s, \hat{s}) \pi(a_{t+1} \mid \hat{s}) P(o_{t+1} \mid s, a_{t+1}) P(s' \mid s, a_{t+1}, o_{t+1}) \mathcal{M}(\hat{s}' \mid \hat{s}, a_{t+1}, o_{t+1})$$

Under our setup the environment update is unifilar at the turn scale. Once (s_t, a_{t+1}, o_{t+1}) are fixed, the next environment state s_{t+1} is unique almost surely. This gives synchronization: as turns accumulate, the belief concentrates along the causal branches consistent with the observations for both the environment and agent states.

Predictive state from data. We can skip latent variables and learn a predictive state representation q_t that summarizes the history by what it implies for future observations under future actions. For finite horizons k ,

$$P(o_{t+1:t+k} \mid a_{t+1:t+k}, z_{\leq t}) = P(o_{t+1:t+k} \mid a_{t+1:t+k}, q_t)$$

In practice it is enough to match the one step case and update q_{t+1} deterministically from (q_t, a_{t+1}, o_{t+1}) . This is the world model view trained by next observation prediction.

These two views meet at the minimal predictive presentation. Among optimal nonlinear predictors of a stationary process, the ε -machine (and with inputs, the ε -transducer) collects histories that induce the same conditional futures into the same causal state. In the finite-state stationary case we care about, and under unifilarity, the history-based predictor is equivalent to a minimal unifilar generator. So the smallest sufficient predictive state you can build from data coincides with the smallest unifilar presentation of the turn process.

What does a strong autoregressive model have to encode to predict our traces. Recall the emission decomposition

$$P(z_{t+1} \mid s_t, \hat{s}_t) = \pi(a_{t+1} \mid \hat{s}_t) \cdot P(o_{t+1} \mid s_t, a_{t+1})$$

A good next turn predictor trained on Z must internalize a predictive latent that is sufficient for after factors. Its internal state $h_{\theta_G}(z_{<t})$ needs to behave like a predictive state q_t : enough information about the history to produce the next action and to forecast the next observation in response to that action. Under unifilarity and stationarity this predictive latent aligns with the causal state of the ε -transducer at the level of turns.

Finally, what does this mean for a transformer trained on sequences of turns. In practice each turn is a tuple $z_t = (a_t, o_t)$. The natural training target is two step: first predict the action, then predict the observation given that action and the history. Concretely the model estimates

$$P(a_{t+1} \mid z_{\leq t}) \quad \text{and} \quad P(o_{t+1} \mid a_{t+1}, z_{\leq t})$$

The internal state right after emitting a_{t+1} is a sufficient statistic of the agent side $\hat{s}_t$ for the purpose of the policy. The internal state right after receiving o_{t+1} is a sufficient statistic of the environment branch s_{t+1} for the purpose of predicting the next observation. This boundary effect also explains a mild separability we observe in practice. A single shared head can support both factors, but the logits factor by conditioning order: action logits depend mostly on the agent slice of the predictive state, observation logits condition further on the realized action and align with the environment slice. That is the concrete story of what a transformer trained on these sequences has learned. It has learned a predictive state that simultaneously supports an action policy and a world model at the level of turns, and it updates that state at the two natural boundaries of the turn, first at action time and then at observation time.

Goal Directedness and State Representation

Until now we only assumed a stationary behavioral policy without any interpretation of its meaning. We now focus on how the agent’s parameters could have come to be in an optimal way. In general an agent’s actions are interpreted as goal directed, that is we read the behavior as an attempt to lead the world toward desired states. We formalize reward as a function on action-conditioned state transitions

$$R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}, \quad (s_t, a_t, s_{t+1}) \mapsto R(s_t, a_t, s_{t+1})$$

which can equivalently be seen as a function on the edges of the transducer graph. For discrete reward classes, this induces an edge coloring.

Given parameters $\theta = (\theta_\pi, \theta_\mathcal{M})$ we interpret the agent's policy as its best attempt to maximize expected return over agent–environment traces Z :

$$J(\theta) = \mathbb{E}_{Z \sim P(Z|\theta)} \left[\sum_{t=1}^T R(s_t, a_t, s_{t+1}) \right]$$

Learning becomes a dynamical process over parameter space, with an update operator $\Phi : \Theta \rightarrow \Theta$ and $\theta_{k+1} = \Phi(\theta_k)$. For example, gradient ascent

$$\Phi(\theta) = \theta + \eta \nabla_\theta J(\theta)$$

or any other optimization scheme. The reinforcement learning problem seeks a fixed point $\theta^* \in \Theta$ satisfying

$$\theta^* = \lim_{k \rightarrow \infty} \Phi^{(k)}(\theta_0) = \arg \max_{\theta \in \Theta} J(\theta)$$

Since $\theta = (\theta_\pi, \theta_\mathcal{M})$, we make explicit that optimization operates over the agent's policy and world model:

$$(\theta_\pi^*, \theta_\mathcal{M}^*) = \arg \max_{\theta_\pi, \theta_\mathcal{M}} \mathbb{E}_{Z \sim P(Z|\theta_\pi, \theta_\mathcal{M})} \left[\sum_{t=1}^T R(s_t, a_t, s_{t+1}) \right]$$

with trajectories generated by $\pi(\cdot | \cdot; \theta_\pi)$ and $\mathcal{M}(\cdot | \cdot; \theta_\mathcal{M})$, while the environment kernels $P(o_{t+1} | s_t, a_{t+1})$ and $P(s_{t+1} | s_t, a_{t+1}, o_{t+1})$ are fixed. In practice we settle for local optima or stationary points with $|\theta_{k+1} - \theta_k| < \epsilon$.

Since we are interested in stationary policies and the environment channel is stationary by construction, we focus on the long-run behavior induced by a given policy when it exists. The short answer to what the agent must understand is well known. By the Good Regulator connection, to implement reward maximizing behavior the agent requires a sufficient representation of the environment hidden states on the portion of state space actually visited by the policy that achieves the optimum. We keep the PSR and belief-state view to relate the agent's internal state $\hat{s}_t$ and the environment state s_t . Since the reward $R(s_t, a_t, s_{t+1})$ is a function of the transition, a world model that predicts the environment's response and updates belief consistently with that transition can also predict reward and support optimal action.

Not all states have nonzero probability under the policy of interest. Given an initial state distribution $P(s_0)$, a policy π may visit only a subset of states. We call this the coverage of the policy. Formally,

$$\mathcal{S}^\pi = \{s \in \mathcal{S} : \exists t \Pr_\pi(s_t = s | P(s_0)) > 0\}$$

When a stationary distribution exists we write

$$d^\pi(s) = \lim_{t \rightarrow \infty} \Pr_\pi(s_t = s)$$

Two related measures quantify breadth and uncertainty of visitation,

$$H_\pi(S_t) = - \sum_s \Pr_\pi(s_t = s) \log \Pr_\pi(s_t = s), \quad H_\pi(S_{t+1} | S_t, A_t)$$

Because the turn process is unifilar at this scale, the conditional entropy $H_\pi(S_{t+1} | S_t, A_t)$ concentrates the uncertainty into the leftover branches consistent with the emitted observation. Intuitively

it counts how many outcome branches remain possible after fixing (S_t, A_t) , up to the identification that different observations can still map to the same next state.

These definitions let us state the target for an optimal world model without referencing reward. The world model with parameters $\theta_{\mathcal{M}}$ is trained to predict the next observation given action and internal state. Let $\hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}})$ be its emission. The objective is

$$\mathcal{L}_{\mathcal{M}}(\theta_{\mathcal{M}}) = \mathbb{E}_{h_t, a_{t+1}} \left[D_{KL}(P(o_{t+1} | a_{t+1}, h_t) | \hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}})) \right]$$

implemented as cross entropy,

$$\mathcal{L}_{\mathcal{M}}(\theta_{\mathcal{M}}) = -\mathbb{E}_{h_t, a_{t+1}, o_{t+1}} [\log \hat{P}(o_{t+1} | a_{t+1}, \hat{s}_t; \theta_{\mathcal{M}})]$$

Given an accurate world model $\theta_{\mathcal{M}}^*$, policy optimization becomes

$$\theta_{\pi}^* = \arg \max_{\theta_{\pi}} \mathbb{E}_{Z \sim P(Z | \theta_{\pi}, \theta_{\mathcal{M}}^*)} \left[\sum_{t=1}^T R(s_t, a_t, s_{t+1}) \right]$$

The principle is simple. Accurate state estimation enables optimal decision making.

Training the world model purely on on-policy data induces a clear importance-sampling relationship between the learned model and the policy. The data distribution is the one induced by π , so the learned predictor is most accurate on transitions with high probability under π and can be weakly constrained on counterfactual branches. In other words, the effective model focuses on $\mathcal{S}^{\pi}$ and on the action branches actually tried. This is why coverage matters. Exploration strategies expand $\mathcal{S}^{\pi}$ and, in stochastic environments, can raise $H_{\pi}(S_t)$, which tightens the world model on more of the transition graph. Uniform random actions and ε -greedy policies guarantee coverage in the sense that every action branch receives nonzero probability, although they do not guarantee maximum state entropy.

Three common regimes make this trade-off explicit.

- Exploration. During training the agent balances exploitation with the discovery of rewarding states, expanding $\mathcal{S}^{\pi}$ and improving constraints on the model.
- Multi-policy learning. When one agent solves multiple tasks in the same environment, overlapping coverage sets make it natural to decouple reward from the world model and reuse a shared predictive representation.
- Goal-directed policies. When reward encodes only the desirability of a final state, the policy family lives on a common transition graph and searches for short paths between nodes of that graph.

When the learning objective requires coverage, the world model must approach a sufficient statistic of the environment, so that its internal state supports any policy defined on the transition graph. The same boundary logic from agentic traces applies. After emitting a_{t+1} the internal state is sufficient for action selection. After ingesting o_{t+1} it is sufficient for predicting future observations and for scoring the colored edge (s_t, a_t, s_{t+1}) .

Learning to Please the Gods from a Drunken Pythia

We conclude with a light-weight toy example inspired by the identity-not channel in computational mechanics. In the channel there are two modes. In the identity mode the output matches the input. In the not mode the output flips the input. The mode itself toggles every turn. The agent chooses

the input each turn, so the evolution is controlled in the simple sense that what happens next depends both on the hidden mode and on the agent’s current act.

On a mountain there is a shrine to two gods, one loyal and one a trickster. The statue has two plates, one marked offering and one marked nothing, and an omen returned after each turn is the only public signal of which god currently guards the shrine. The loyal god returns offering with blessing and nothing with curse, while the trickster returns nothing with blessing and offering with curse; after each turn the satisfied guardian departs and the other takes the watch. An old and drunken pythia serves the shrine by choosing between offering and nothing at random. She has done this for decades and still does not understand the pattern. To make matters worse, after one hundred turns the gods go on vacation for a week, the shrine falls silent, and when it returns it simply begins glowing again, ready for offering, and there is no way to know which guardian has come back to the post. She no longer knows whether blessing means the loyal god who mirrors the act or the trickster who inverts it, so she gave up trying to reason it out. We study the problem from the perspective of her disciple, who has observed this behavior for years and will soon take over. She wants to please the gods as well as possible and asks what can be inferred from the drunken policy to recover the correct strategy.

We use the shrine as a toy environment. Each turn is a pair $z_t = (a_t, o_t)$ with a_t the act (offering or nothing) and o_t the omen (blessing or curse). The shrine has a hidden two-mode state $s_t \in \{0, 1\}$, read as loyal versus trickster. Saying the update is controlled means the next omen is produced from the current act together with the current guardian, and the guardian for the next turn is determined by what just happened. In compact form,

$$s_t = \mathbb{1}[a_{t-1} = o_{t-1}], \quad o_t = a_t \oplus s_t.$$

Here $s_t = 0$ is loyal and $s_t = 1$ is trickster, so $o_t = a_t$ under loyal and $o_t \neq a_t$ under trickster. This also implies the period-2 alternation $s_t = \neg s_{t-1}$ since $o_{t-1} = a_{t-1} \oplus s_{t-1}$.

Inputs alone do not suffice and outputs alone do not suffice. The joint pair (a_{t-1}, o_{t-1}) fixes the mode for the step, and once the turn (a_t, o_t) is realized the environment updates unifarlarly. This matches the agentic trace view above. The agent proposes a_t . The shrine emits o_t given s_t and a_t . The minimal predictive latent the model must carry is the phase s_t .

To keep runs independent we add a renewal. After one hundred turns the gods leave for a week and the shrine falls silent. When the shrine restarts it simply begins glowing again, ready for offering, and the guarding mode is drawn as $s_0 \sim \text{Bernoulli}(1/2)$. That draw sets the phase for the new epoch. Within an epoch the one step joint history (a_{t-1}, o_{t-1}) together with the current act is sufficient to predict o_t and to update s_t .